COURSE-BASED PROJECT DOCUMENTATION | DOC-12

Python Backend & ctypes FFI

Foreign Function Interface, In-Memory Invocation, & Fallback

1. Foreign Function Interface (ctypes)

Python's standard ctypes module allows calling functions in native DLLs directly within the Python process memory space. This eliminates the operating system overhead of repeatedly spawning subprocesses.

2. Backend Initialization (ParserBackend)

```
import ctypes

class ParserBackend:
    def __init__(self):
        self.use_dll = False
        if os.path.exists(DLL_PATH):
            try:
                self.dll = ctypes.CDLL(DLL_PATH)
                self.dll.check_syntax.argtypes = [
                    ctypes.c_char_p,   # filepath (UTF-8 bytes)
                    ctypes.c_char_p,   # output message buffer
                    ctypes.c_int       # buffer length
                ]
                self.dll.check_syntax.restype = ctypes.c_int
                self.use_dll = True
            except Exception:
                self.use_dll = False
```

3. In-Memory Code Checking (check_code_string)

To check code typed live into the editor, check_code_string() safely writes the text buffer to a temporary file using Python's tempfile module, invokes check_file(), and deletes the temporary file in a finally block.

4. Subprocess Fallback

If javacheck.dll cannot be loaded (due to permissions or environment constraints), ParserBackend automatically falls back to invoking javacheck.exe via subprocess.run(), ensuring continuous operation.